# Lightweighted Hyperspectral Image Classification Network by Progressive Bi-Quantization

Wei Wei, *Senior Member, IEEE*, Chongxing Song, Lei Zhang, *Member, IEEE*, and Yanning Zhang, *Senior Member, IEEE*

***Abstract*— Convolutional neural network (CNN) has shown its powerful ability for hyperspectral image (HSI) classification, which however, is difficult to deploy on resource-limited or low-latency platforms due to its parameter and computation redundancy. Though binary neural network (BNN) has attracted attention for its extreme compressing and speeding up ability by binarizing both weights and activations, it has rarely been explored for HSI classification. In this study, we elaborately design a BNN with good performance for HSI classification task. Specifically, an adaptive gradient scale module is proposed to flexibly modify the gradient during training stage to better optimize the BNN and does not add any extra computation for inference. Furthermore, a curriculum learning-based progressive binarization strategy is utilized to improve the performance. Compared with the existing BNN works, our method can increase the HSI classification accuracy by a large margin while maintaining the compressing ratio. Abundant experiments on three datasets demonstrate the effectiveness of the proposed method.**

***Index Terms*— Binary neural network (BNN), hyperspectral image (HSI) classification, progressive quantization.**

## I. Introduction

HYPERSPECTRAL image (HSI) is a kind of image that contains hundreds or thousands of bands, and each pixel has a continuous spectrum. Benefiting from its rich spectral information that can discriminate the subtle differences among substances, HSI has been widely used in various remote-sensing-related applications [1], [2], [3].

HSI classification aims to predict the class of each pixel within HSI. Existing HSI classification methods can be roughly divided into traditional methods and deep learning-based ones. Traditional methods usually consist of a feature-extracting module and a feature-classifying module. Feature-extracting module [1], [4], [5], [6], [7], [8] reduces the dimension of features and obtains representative features while maintaining the separability among various classes. Feature-classifying module [9], [10], [11] classifies the extracted low-dimensional features into different class space. However, due to the limited representation ability caused by the shallow and handcrafted feature, it is hard to obtain satisfactory HSI classification performance directly utilizing the traditional methods.

In recent years, deep learning has demonstrated its superiority in a set of remote-sensing-related tasks due to its powerful feature representation ability [12], [13], [14], [15]. As a result, different deep learning-based methods, especially convolutional neural networks (CNNs), have been widely utilized for HSI classification and obtained pleasing performance. For example, [16] proposes a 1-D-CNN which consists of several layers to extract spectral features for HSI classification. [17] first uses principle component analysis (PCA) to reduce the dimension and then applies a 2-D-CNN to utilize the spatial information of neighbor pixels for classification. Yue et al. [18] further trains a 3-D-CNN to exploit both spectral and spatial information for HSI classification simultaneously. Based on CNN, residual structure [19] is introduced to mitigate the gradient vanishing problem when network goes deeper, and densely connected structure [20] is utilized to make use of both shallow and deep features. In addition, multiscale convolution [21], deformable convolution [22], dilated convolution [23], nonlocal block [24], and attention structure [25], [26], [27], [28] are proposed sequentially to further improve the performance of HSI classification.

Although obtaining pleasing HSI classification performance, deep learning-based methods often suffer from over-parametrization problem, which inevitably results in expensive memory consumption and computation. As a result, it is hard to deploy such models directly on devices with strict latency requirements and limited resources (e.g., device on airborne platform). To address this problem, suitable model compressing and accelerating methods are desired, from which the parameters and running time can be significantly reduced. Though binary neural network (BNN) is an extreme quantization method and has tiny memory overhead and fast inference speed by binarizing both weights and activations

Manuscript received 15 October 2022; revised 3 December 2022; accepted 11 January 2023. Date of publication 16 January 2023; date of current version 26 January 2023. This work was supported in part by the National Natural Science Foundation of China under Grant 62071387, Grant 62101454, and Grant U19B2037; in part by the Shenzhen Fundamental Research Program under Grant JCYJ20190806160210899; and in part by the Fundamental Research Funds for the Central Universities. *(Corresponding author: Lei Zhang.)*

Wei Wei is with the Shaanxi Provincial Key Laboratory of Speech and Image Information Processing and the National Engineering Laboratory for Integrated Aero-Space-Ground-Ocean Big Data Application Technology, School of Computer Science, Northwestern Polytechnical University, Xi'an 710072, China, and also with the Research and Development Institute, Northwestern Polytechnical University in Shenzhen, Shenzhen 518031, China (e-mail: weiweinwpu@nwpu.edu.cn).

Chongxing Song, Lei Zhang, and Yanning Zhang are with the Shaanxi Provincial Key Laboratory of Speech and Image Information Processing and the National Engineering Laboratory for Integrated Aero-Space-Ground-Ocean Big Data Application Technology, School of Computer Science, Northwestern Polytechnical University, Xi'an 710072, China (e-mail: 927879551@mail.nwpu.edu.cn; zhanglei211@mail.nwpu.edu.cn; ynzhang@nwpu.edu.cn).

Digital Object Identifier 10.1109/TGRS.2023.3237668

1558-0644 © 2023 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 16:03:31 UTC from IEEE Xplore. Restrictions apply.

into $+1$ or $-1$, the research on HSI is rarely explored and there is still a lack of suitable BNN for HSI classification.

In this study, we explore a BNN-based HSI classification method to achieve the tradeoff between classification accuracy and inference speed. Specifically, we first construct a baseline BNN for HSI classification utilizing spectral information, then we design an adaptive gradient scale module to adaptively change the gradient during training stage so as to better optimize the BNN. Furthermore, considering that BNN is vulnerable and susceptible to the initialization, feeding samples difficult to be classified to BNN at the initial stage of training may make the model trap into the suboptimal point. To address this problem, we take inspiration from the idea of curriculum learning and progressively enhance the capacity of BNN during training to mitigate the quantization error. Experimental results demonstrate the effectiveness of the proposed method.

In summary, the main contribution of our method is as follows.

1) We systematically test and compare the performance of the existing BNNs for HSI classification. Based on this, we elaborately design a BNN with high accuracy for HSI classification.
2) We propose an adaptive gradient scale module that only exists during training stage to flexibly modify the gradient so that the BNN can be better optimized without adding extra computation for inference.
3) A curriculum learning-based progressive quantization strategy is further proposed to improve the performance of BNN for HSI classification.
4) Experimental results on three datasets demonstrate that the proposed method achieves the state-of-the-art performance compared with any other competing methods.

The remainder of this article is organized as follows. Section II introduces the related works of this study. Section III presents the proposed BNN with the adaptive gradient scale module and the proposed progressive binarization strategy. Section IV summarizes the experimental results, ablation study, and analysis on three datasets. Finally, Section V concludes the article.

## II. Related Works

In this section, we first introduce the works related to HSI classification task. Then in detail we introduce the recent progress of BNN.

### A. HSI Classification

HSI classification aims to predict the class of each pixel within HSI, which can be roughly divided into traditional methods and deep learning-based ones.

*1) Traditional Methods:* These methods usually extract features first and then classify them via a predefined classifier. Prasad and Bruce [1], Licciardi et al. [4], and Villa et al. [5] utilize PCA or independent component analysis (ICA), etc., to project the spectral information in the original high-dimensional space into the low-dimensional space to extract representative features. Besides, some feature extraction methods based on spectral–spatial joint information, such as composite kernel learning [6], morphological profiles [7], and Gauss–Markov random fields [8] are proposed to extract more informative features. Then with these features, support vector machines (SVMs) [9], decision tree [10], and random forest [11] are usually utilized to construct a classifier to separate the obtained features into different classes.

*2) Deep Learning-Based Methods:* Traditional methods often suffer from unsatisfactory classification performance due to its shallow structure. In recent years, deep learning-based methods, especially CNN, have been widely applied in HSI classification task and achieved excellent performance due to its deep feature extraction ability. For example, [16] utilizes a 1-D-CNN to extract the spectral features; [17] and [18] construct a 2-D-CNN and a 3-D-CNN to extract spectral–spatial joint features. Furthermore, [19] designs a deep CNN with residual structure for HSI classification to mitigate the gradient vanishing problem and enhance the classification accuracy. Yang et al. [20] proposes a two-branch densely connected structure to extract hierarchical spectral features and spatial features, respectively. Gong et al. [21] embeds multiscale convolution into network to extract more representative features. Zhu et al. [22] applies deformable convolution to adaptively sample important pixels and Pooja et al. [23] applies dilated convolution on features to enlarge the receptive field. Wang et al. [24] introduces nonlocal module into HSI classification network to model the long-range dependencies between pixels and capture more informative features. Mou and Zhu [25] plugs spectral attention module into network to adaptively emphasize important spectral bands. Sun et al. [26] and Hang et al. [27] design spectral–spatial attention to recalibrate the importance of features elementwisely. Besides, Zheng et al. [28] proposes a rotation-invariant attention module to alleviate the damage of spatial rotation to the HSI classification model.

*3) Binary Neural Network:* A common problem faced by deep learning-based methods is the overparameterized network, which seriously hinders the application of HSI classification algorithms on resource-limited or low-latency platforms. Commonly utilized methods for obtaining lightweight CNN include quantization [29], [30], pruning [31], [32], lightweight network structure design [33], [34], low-rank factorization [35], and knowledge distillation [36]. Among these methods, BNN [37] is an extreme form of quantization whose weights and activations can be represented by two values (i.e., $+1$ and $-1$). In this case, convolution can be implemented efficiently using exclusive NOR (XNOR) and bit-counting operators so that BNN can theoretically achieve $\sim 58\times$ speed up and $\sim 32\times$ memory saving compared to its counterpart full-precision network.

However, the existence of binarization function (i.e., the sign function) leads to a performance difference between BNN and its counterpart full-precision network, which is mainly caused by two aspects. On the one hand, the representation ability of sign function leads to large quantization errors of activations and weights in the forward-propagation process. On the other hand, the sign function is nondifferentiable. To enable the network to be trained, STE [38]

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 16:03:31 UTC from IEEE Xplore. Restrictions apply.

(i.e., straight-through estimator) is usually utilized to acquire the approximate gradient in the back-propagation process, which inevitably brings gradient error. Several works have been proposed to address these two problems, respectively. For example, to reduce the quantization error, XNOR-Net [39], XNOR-Net++ [40], and Real2Bin [41] introduce a scaling factor for binarized weights or activations to better approximate the full-precision counterpart. GroupNet [42] proposes to aggregate multiple base binary convolution units or groups to approximate the full-precision ones. Bi-Real Net [43] proposes an identity shortcut to connect the real-valued activations with the activations of the next layer before binarizing to enrich the numerical diversity of feature maps and enhance the representation ability of the model. ReAct-Net [44] further utilizes a learnable shift parameter for activation function to explicitly learn and reshape the activations distribution. IR-Net [45] reduces the quantization error of both weights and activations by balancing and standardizing the weight before binarizing from the view of maximizing information entropy. Besides, to reduce the gradient error, Bi-Real Net [43] proposes to use a piecewise polynomial function to approximate the sign function in the backward-propagation process. Information retention network (IRNet) [45], differentiable soft quantization (DSQ) [46] reduce the gradient error by progressively approaching the sign function in the back-propagation stage of the training process. Zhuang et al. [47] designs a full-precision auxiliary module to provide an extra gradient path during back-propagation to promote the convergence of quantized model. Element-wise gradient scaling (EWGS) [48] proposes an EWGS strategy to adaptively scale up or down each gradient element based on quantization error.

Though very important for real applications, the study on BNN for HSI classification is still a new topic. Mei [49] designs a 1-D-CNN with binarized weights but 2-bit step-style quantized activations for HSI classification. Though it greatly reduces the network parameters, it is not exactly a BNN because 2-bit is utilized for activations within the network.

In this study, we systematically test and compare the performance of the existing BNNs for HSI classification. In addition, we propose a new BNN with both binarized weights and activations for HSI classification. The proposed BNN can 1) adaptively scale and correct the gradient in the training stage without adding extra parameters storage and computation for inference, 2) gradually increase its classification ability to avoid trapping into suboptimal solution. The proposed method can compress and accelerate the HSI classification model while minimizing the loss of accuracy.

## III. Proposed Method

Denote the HSI as $\mathcal{H} \in \mathbb{R}^{h\times w\times d}$ where $h$, $w$, and $d$ represent the height, width, and number of spectral bands, respectively. HSI classification aims to assign a class label $y_i \in \{1, 2, \ldots, C\}$ to each hyperspectral pixel vector $x_i \in \mathbb{R}^d$, where $C$ is the number of classes. We define the labeled training dataset as $\mathcal{D}_S = \{(x_i, y_i), i = 1, 2, \ldots, N\}$ and the unlabeled test dataset as $\mathcal{D}_T = \{x_i, i = N+1, N+2, \ldots, N+M\}$, with $N$ and $M$ being the size of training dataset and test dataset, respectively. HSI classification task aims to utilize $\mathcal{D}_S$ to learn the mapping relationship from $x_i$ to $y_i$ and finally generalize it to $\mathcal{D}_T$. At present, CNNs exhibit good performance in capturing this relationship, which however, are difficult to be deployed on resource-limited or low-latency platforms since CNNs demand a large amount of parameters and calculation. Though BNN provides an effective way to deal with this problem, the research of BNN on HSI is rarely studied. In the following, we will first introduce the standard BNN and its limitation, then we introduce the proposed BNN method.

### A. BNN Revisited

*1) Forward Propagation:* Both weight parameters and activations in BNN are binarized with the sign function as

$$w_b = \text{Sign}(w_r) = \begin{cases} +1, & \text{if } w_r \geq 0 \\ -1, & \text{otherwise} \end{cases}$$
$$a_b = \text{Sign}(a_r) = \begin{cases} +1, & \text{if } a_r \geq 0 \\ -1, & \text{otherwise} \end{cases} \tag{1}$$

where $w_r$ and $w_b$ denote the real-valued and binarized weights, $a_r$ and $a_b$ denote the real-valued and binarized activations, respectively. Therefore, full-precision convolution can be approximated by binarized convolution as

$$a_r \otimes w_r \approx a_b \otimes w_b \tag{2}$$

where $\otimes$ denotes the convolution operator.

*2) Backward Propagation:* Since the magnitude of the gradient with respect to the weights is usually too small to make the sign of weights flip, (i.e., from $+1$ to $-1$ or from $-1$ to $+1$), to better optimize the BNN, the latent full-precision weights are usually stored and updated during training stage to capture the effect of the subtle gradient.

Another area for improvement is that the model is not trainable due to the nondifferentiable sign function. To tackle this problem, a common practice is bypassing the gradient of sign function using straight-through estimator (STE)

$$\frac{\partial L}{\partial w_r} \approx \frac{\partial L}{\partial w_b} * 1_{|w_r|\leq 1}$$
$$\frac{\partial L}{\partial a_r} \approx \frac{\partial L}{\partial a_b} * 1_{|a_r|\leq 1} \tag{3}$$
$$w_r' = w_r - \alpha * \frac{\partial L}{\partial w_r} \approx w_r - \alpha * \frac{\partial L}{\partial w_b}. \tag{4}$$

The usage of STE is shown as (3) and the latent full-precision weights update criteria is shown as (4), where $L$ is loss of output, $\alpha$ is the learning rate, $w_r'$ is the updated weight, and $1_{\{.\}}$ is the indicator function that returns to 1 if the condition in {} is true and returns 0 otherwise.

Though BNN provides a way to construct a lightweight CNN, it is rarely considered for HSI classification. In this study, we build an HSI-oriented BNN method from the following two aspects. First, to solve the problem of serious gradient error caused by the conventional STE in the back-propagation process, we design an adaptive gradient scaling (AGS) module to recalibrate the gradient. Second, considering that pixels

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 16:03:31 UTC from IEEE Xplore. Restrictions apply.

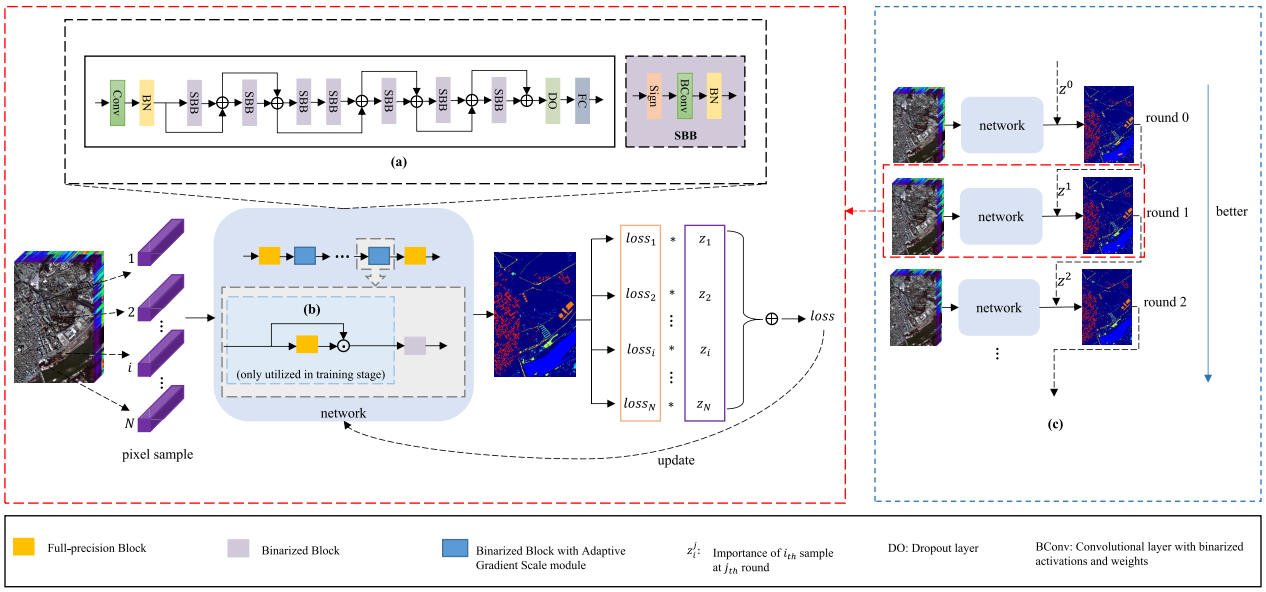



Fig. 1. General training flowchart of our method.

within HSI show different difficulties to be classified, equally treating each pixel within BNN for HSI classification may cause the optimization of BNN to deviate from the optimal solution. To address this problem, a curriculum learning-based progressive binarization strategy is utilized to improve the performance of BNN.

### B. Overall Framework of Our Method

The general training flowchart of our method is shown as Fig. 1. The overall framework mainly consists of the constructed basic BNN, the adaptive gradient scale module, and the curriculum learning-based progressive binarization strategy.

We first build the network based on the 1-D-CNN structure following [49]. Within this network, we remove the activations binarization function of the last layer following the general practice of the BNN in the classification task. Besides, we add several real-valued shortcuts to maintain the representation ability in the forward-propagation stage and provide an extra full-precision gradient flow in the back-propagation stage [43]. The detailed structure of the constructed basic BNN is shown as Fig. 1(a). It can be seen that the overall architecture consists of eight convolutional layers and all convolutional layers except the first layer are binarized. And each convolutional layer is followed by a BatchNorm layer. In addition, there are a dropout layer with drop-rate 0.5 and a full-precision fully connected layer for classification at the end of the network. It is worth noting that the number of input and output channels of the third binary convolutional layer is not equal in [49], so a corresponding residual connection spans two SBB modules in Fig. 1(a).

Afterward, an adaptive gradient scale module shown as Fig. 1(b) is plugged into the binarized part of the network during training stage to flexibly modify the back-propagated gradient. After getting the well-trained model, the adaptive scale module will be discarded.

Besides, an optimization strategy based on curriculum-learning strategy shown as Fig. 1(c) is proposed. It enables pixel samples to be trained from easy to difficult according to the classification difficulty and iteration time, which is accomplished by giving different importance to each pixel within HSI. During training stage, we optimize the network weight parameters and importance of samples for $r$ rounds. And only one round is needed for inference. In the following, we will detailedly introduce the proposed adaptive gradient scale module and the curriculum learning-based progressive quantization strategy.

### C. Adaptive Gradient Scale Module

The suboptimal performance of STE in the back-propagation process can be attributed to two aspects. On the one hand, the first-order derivative of sign function is an impulse function, while the first-order derivative is a gated function when using STE, which results in an inaccurate approximation. Some following works have been proposed for better approximation, such as Bi-Real Net [43] that adopts a piecewise polynomial function. On the other hand, STE propagates the same gradient without considering the difference of forward quantization error among elements. Though the piecewise polynomial function mitigates this defect by giving a smaller gradient to the elements with smaller quantization error, it is a fixed form, which cannot adapt to the data and thus still be suboptimal.

In this section, we propose an adaptive gradient scale module to address this problem. Specifically, during training, we add a data-driven module that includes several layers before sign function of activations binarization. To maintain the same effect as when only using the sign function in the forward stage, we force the output of the module to be positive. By doing so, it is equivalent to multiplying each gradient element with an adaptive positive scale in the backward-propagation stage without affecting the forward process. More importantly, the module can be removed during

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 16:03:31 UTC from IEEE Xplore. Restrictions apply.

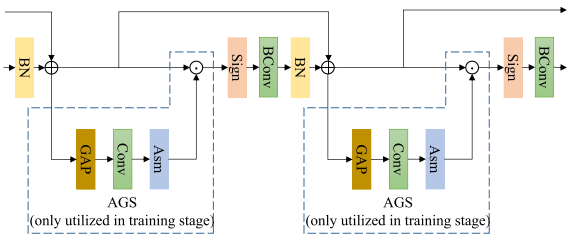



Fig. 2. Binarized block with AGS module. GAP denotes global average pooling layer. Asm denotes Sigmoid function with two adjustable parameters. The AGS module only exists during training and is discarded at inference stage.

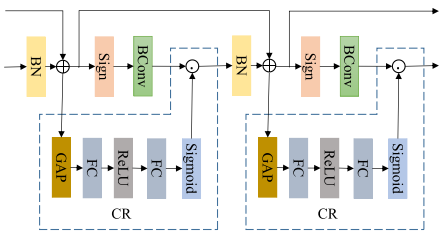



Fig. 3. Binarized block with CR module. GAP denotes global average pooling layer.

inference stage to avoid increasing additional parameters and computation.

The detailed network structure of the proposed adaptive gradient scale module is shown as Fig. 2. The full-precision feature map is first passed through a global average pooling layer to gather the information, then a 1-D-Conv layer is utilized for the information interaction between the feature maps of each channel. Finally, the positive scale factors differ in channel granularity are obtained by an adjustable sigmoid function formulated as $(1/(1+\exp(-c_1 * x))) + c_2$, where $c_1$ and $c_2$ are adjustable parameters that vary with datasets. After obtaining the scale factors, we multiply them by the activations before passing the sign function to get scaled activations. The process can be formulated as

$$a_r \otimes w_r \approx \text{Sign}(a_r \odot G(a_r; \theta_G)) \otimes w_b = a_b \otimes w_b \tag{5}$$

where $\odot$ denotes the element-wise multiplication and $G$ is the adaptive gradient scale module parameterized by $\theta_G$. During back-propagation stage, the existence of AGS module will affect the propagation of gradient as

$$\frac{\partial L}{\partial a_r} \approx \frac{\partial L}{\partial a_b} * \left( G(a_r; \theta_G) + \frac{\partial G(a_r; \theta_G)}{\partial a_r} * a_r \right) * 1_{|a_r \odot G(a_r; \theta_G)| \le 1}. \tag{6}$$

When the gradient is back-propagated from the tail of the network, the gradient concerning the activations of current layer is firstly scaled, then the gradient concerning the activations and weights of the shallow layer in the front will be affected and scaled sequentially.

*Difference From Data-Driven Channel Rescaling (CR):* In Real2Bin [41], the full-precision activations before binarization are utilized for channel wisely rescaling the activations after the binary convolution during both training and inferencing phase. The detailed structure is shown as Fig. 3. This process can be formulated as

$$a_r \otimes w_r \approx (a_b \otimes w_b) \odot G(a_r; \theta_G) \tag{7}$$

where $G$ is the CR module parameterized by $\theta_G$. By comparing (5) and (7), we can find that the binarized convolution results of these two methods are different. Specifically, our method does not change the binarized convolution result in the forward-propagation process, while the comparison method Real2Bin is equivalent to scaling the binarized convolution result. Besides, the network design between ours and the compared scale module is different and our design is more lightweight due to discarding several fully connection layers. More importantly, benefiting from the consistent forward result of our method, we can remove it during inference stage without adding any additional parameters and computation. However, Real2Bin still needs to maintain the scaling module with additional parameters and computation cost when inferencing to avoid the forward gap between the training and inferencing phase.

### D. Curriculum Learning-Based Progressive Quantization

Curriculum learning strategy optimizes the model by simulating the cognitive process of people. It advocates that the model should start to learn from the easy samples and gradually advance to the complex ones. On the one hand, HSI is composed of a large number of pixels with different classes, and the classification difficulty of each pixel is different. The conventional practice of feeding pixel samples into the network for training without distinguishing the difficulty may cause the model to trap into the suboptimal solution. On the other hand, previous studies [40] have shown that the performance of BNN is susceptible to the initialization and it is challenging to get a well-optimized BNN by binarizing the weights and activations simultaneously during training stage. To tackle this problem, some works [40], [41], and [50] have proposed the progressive binarization strategy, that is, the model with binarized activations and full-precision weights is taken as the pretrained model, or they gradually binarize the activations or weights from part to whole during training.

Inspired by these two aspects, we design a progressive binarization method with sample granularity based on curriculum learning for HSI classification. Specifically, in the conventional HSI classification–optimization process, all HSI pixel samples are given the same importance and the weight term of criterion loss belonging to each pixel sample is the same. The description above can be formulated as

$$\min_{\{\theta^B\}} -\frac{1}{N} \sum_{i=1}^{N} \sum_{j=1}^{C} y_{i,j} \log\left(F_j\left(x_i; \theta^B\right)\right) \tag{8}$$

where $\{x_i, y_i\}_{i=1}^N$ denotes the training dataset with $C$ classes, $F$ denotes the BNN with network parameters $\theta^B$, $F_j$ denotes the predicted probability that the pixel sample $x_i$ belongs to class $j$, and $y_{i,j}$ denotes whether the ground truth of the pixel sample $x_i$ belongs to class $j$.

In contrast, we regard the pixel samples with larger classification error as the more difficult ones. In the early stage of

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 16:03:31 UTC from IEEE Xplore. Restrictions apply.

training, we give a bigger weight to the loss term of simple pixel samples so that the BNN can focus on learning simple knowledge and obtain better initialization. As the training progresses, the classification ability of the BNN continues to increase, so we can gradually enhance the weight term of difficult pixel samples to further train the BNN and improve its capability. In other words, what we expect is that the importance value of the sample increases from 0 to 1 during training, and the more easily the pixel sample is classified correctly, the faster the importance value increases. The description above can be formulated as

$$\min_{\{\theta^B, z_i\}} -\frac{1}{N}\sum_{i=1}^{N} z_i \sum_{j=1}^{C} y_{i,j}\log\left(F_j\left(x_i;\theta^B\right)\right) + H(z_i)$$
$$\text{s.t. } \forall i, \quad 0 \le z_i \le 1 \tag{9}$$

where $z_i$ denotes the pixel sample-wise importance, and $H(z_i)$ denotes the penalty function over $z_i$ which controls the change trend of the importance. We adopt the penalty function form $H$ and the alternative minimization optimization scheme in adaptive importance learning (AIL) [51] to solve the problem in (9). The final optimization problem can be formulated as

$$\min_{\{\theta^B, z_i\}} -\frac{1}{N}\sum_{i=1}^{N} z_i \sum_{j=1}^{C} y_{i,j}\log\left(F_j\left(x_i;\theta^B\right)\right) + H\left(z_i, z_i'\right)$$
$$\text{s.t. } H\left(z_i, z_i'\right) = \sum_{j}^{N}\left(z_i - z_i'\right)\left(\ln\left(\frac{z_i - z_i'}{\beta}\right) - 1\right)$$
$$\forall i, \quad 0 \le z_i' \le z_i \le 1 \tag{10}$$

where $\beta$ denotes the hyperparameter and $z_i'$ denotes the pixel sample-wise importance value in the last iteration. We can optimize $z_i$ and $\theta^B$ alternatively for several rounds. When the importance value $z_i$ is fixed, we minimize the loss item $-(1/N)\sum_{i=1}^{N} z_i \sum_{j=1}^{C} y_{i,j}\log(F_j(x_i;\theta^B))$ using the conventional back-propagation and gradient-descent algorithm. When network parameters $\theta^B$ is fixed, benefiting from the elaborately designed function $H$, the optimal solution of importance value $z_i$ in (10) exactly right satisfies the desired change trend. This means the importance of sample will increase gradually during training. The more difficult the sample to be classified, the classification loss will be larger, and thus the growth of sample importance will be slower. As a result, the importance of simple samples will increase to 1 faster than that of difficult ones, which means the BNN can learn from simple to difficult. In addition, since the importance will not change after increasing to 1, simple pixel samples still participate in loss calculation in the last few rounds of training. For detailed proof, please refer to AIL [51]. Besides, it is worth noting that we can randomly initialize $z_i$ or obtain it from the classification loss of a pretrained full-precision model at the first round. To save the training cost, we choose the former.

## IV. Experiments and Analysis

### A. Experimental Settings

*1) Datasets:* To demonstrate the effectiveness of the proposed method, we conduct experiments on three benchmark HSI classification datasets including Pavia University, Salinas, and Houston.

TABLE I
NUMBER OF TRAINING AND TEST SAMPLES IN THE PAVIAU DATASET

| Class | Name | Training | Test |
|---|---|---|---|
| 1 | Asphalt | 50 | 6581 |
| 2 | Meadows | 50 | 18599 |
| 3 | Gravel | 50 | 2049 |
| 4 | Trees | 50 | 3014 |
| 5 | Painted metal sheets | 50 | 1295 |
| 6 | Bare Soil | 50 | 4979 |
| 7 | Bitumen | 50 | 1280 |
| 8 | Self-Blocking Bricks | 50 | 3632 |
| 9 | Shadows | 50 | 897 |
| | Total | 450 | 42326 |

TABLE II
NUMBER OF TRAINING AND TEST SAMPLES IN THE SALINAS DATASET

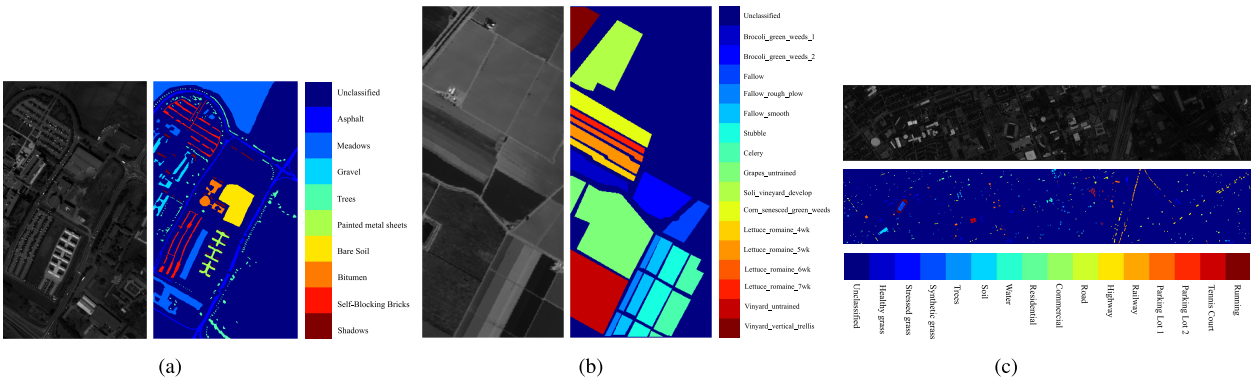
| Class | Name | Training | Test |
|---|---|---|---|
| 1 | Brocoli_green_weeds_1 | 50 | 1959 |
| 2 | Brocoli_green_weeds_2 | 50 | 3676 |
| 3 | Fallow | 50 | 1926 |
| 4 | Fallow_rough_plow | 50 | 1344 |
| 5 | Fallow_smooth | 50 | 2628 |
| 6 | Stubble | 50 | 3909 |
| 7 | Celery | 50 | 3529 |
| 8 | Grapes_untrained | 50 | 11221 |
| 9 | Soli_vineyard_develop | 50 | 6153 |
| 10 | Corn_senesced_green_weeds | 50 | 3228 |
| 11 | Lettuce_romaine_4wk | 50 | 1018 |
| 12 | Lettuce_romaine_5wk | 50 | 1877 |
| 13 | Lettuce_romaine_6wk | 50 | 866 |
| 14 | Lettuce_romaine_7wk | 50 | 1020 |
| 15 | Vinyard_untrained | 50 | 7218 |
| 16 | Vinyard_vertical_trellis | 50 | 1757 |
| | Total | 800 | 53329 |

**Pavia University Dataset** is a HSI dataset gathered by the ROSIS-3 sensor over the city of Pavia, Italy. The geometric resolution is 1.3 m and the image consists of $610 \times 610$ pixels with 115 spectral bands. An image consisting of $610 \times 340$ pixels is obtained after removing a dark region of $610 \times 270$. The image ground truth can be divided into nine classes and the number of samples of each class for training and testing is listed as Table I.

**Salinas Scene Dataset** is a HSI dataset collected by the AVIRIS sensor over Salinas Valley, California. Its geometric resolution is 3.7 m and the image consists of $512 \times 217$ pixels with 224 spectral bands. About 20 water absorption bands are discarded in experiments. Its ground truth contains 16 classes and the number of samples of each class for training and testing is listed as Table II.

**Houston Dataset** is a HSI dataset collected over the University of Houston campus. The geometric resolution is 2.5 m and the image consists of $349 \times 1905$ pixels with 144 spectral bands. Its ground truth contains 15 classes and the number of samples of each class for training and testing is listed as Table III.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 16:03:31 UTC from IEEE Xplore. Restrictions apply.




(a) (b) (c)

Fig. 4. Visualization of the representative band and the class labels of three utilized datasets. (a) PaviaU. (b) Salinas. (c) Houston.

TABLE III
NUMBER OF TRAINING AND TEST SAMPLES IN THE HOUSTON DATASET

| Class | Name | Training | Test |
|---|---|---|---|
| 1 | Healthy grass | 50 | 1201 |
| 2 | Stressed grass | 50 | 1204 |
| 3 | Synthetic grass | 50 | 647 |
| 4 | Trees | 50 | 1194 |
| 5 | Soil | 50 | 1192 |
| 6 | Water | 50 | 275 |
| 7 | Residential | 50 | 1218 |
| 8 | Commercial | 50 | 1194 |
| 9 | Road | 50 | 1202 |
| 10 | Highway | 50 | 1177 |
| 11 | Railway | 50 | 1185 |
| 12 | Parking Lot 1 | 50 | 1183 |
| 13 | Parking Lot 2 | 50 | 419 |
| 14 | Tennis Court | 50 | 378 |
| 15 | Running Track | 50 | 610 |
| | Total | 750 | 14279 |

The gray images of the representative band and class labels of all datasets are shown as Fig. 4.

*2) Implementation Details:* Following [49], the value of each pixel is updated to the mean of the $9 \times 9$ neighborhood to capture the spatial information in the data preprocessing phase. All experiments are implemented using Pytorch 1.8.1 framework equipped with CUDA10.2 on a hard platform with a 1080Ti GPU. We train all methods for 200 epochs with mini-batch size of 64 using the Adam optimizer, and the initial learning-rate is set to 1e-2 and is decayed by 0.1 at the 100th and 150th epoch. In particular, in our method, the number of alternate optimization rounds is set to 10 and the hyper-parameter $\beta$ is set to 0.15. All experiments are run for ten times with different random seeds to alleviate the effects of random initialization on the performance.

*3) Competitors and Evaluation Metrics:* Considering that the BNN applied in HSI classification task is rare, we transfer some quantization methods that perform well in the conventional image classification task to HSI classification task as our comparison methods. They include XNOR [39], XNOR++ [40], Real2Bin [41], GroupNet [42], Bi-Real Net [43], ReactNet [44], IRNet [45], EWGS [48], Mei [49], rectified clamp unit (ReCU) [52], and learned step size quantization (LSQ) [53]. Among them, XNOR, XNOR++, Bi-Real Net, ReactNet, ReCU, IRNet, and EWGS are BNNs with binarized weights and activations. GroupNet aggregates multiple binary convolutional bases (we set the number of bases to 5 in experiments) to approximate the counterpart full-precision convolution and it is similar to multibit convolution. Mei is a quantization method with 1-bit weights and 2-bit activations and LSQ is a quantization method with 2-bit weights and activations. For a fair comparison, except that XNOR and XNOR++ have no real-valued shortcuts according to their original structure settings, we keep the structure of all methods the same as that in Fig. 1(a). We adopt three evaluation metrics including the overall accuracy (OA) of all classes, the average accuracy (AA) of each class, and the Kappa (Kappa) coefficient.

### B. Performance Comparison on Different Datasets

*1) Accuracy Comparison:* As shown in Tables IV–VI, our method achieves the state-of-the-art performance compared to other BNN methods on all datasets. Compared with the baseline Bi-Real Net, our method improves the OA by 9.6%, 1.2%, and 3.4% on PaviaU, Salinas, and Houston dataset, respectively. More importantly, our method also performs better than GroupNet which is similar to the multibit quantization, and Mei in which 2-bit activations quantization is adopted. For example, the OA of the proposed method exceeds that of GroupNet and Mei by 6.56% and 8.51%, respectively, on PaviaU dataset. Compared to LSQ with 2-bit weights and activations, our method can even surpass it by 0.78% on PaviaU dataset. Besides, we can see that our method is even better than the counterpart full-precision network on PaviaU dataset. This may be because quantization method is more robust and plays a role in regularization, which is more prominent when the dataset size is small and the model is prone to over-fitting. In addition, it is worth noting that BNNs without real-valued residual shortcuts, namely XNOR and XNOR++, have a large accuracy gap compared with other BNNs, which demonstrates the importance of real-valued residual shortcuts in reducing quantization error and accuracy loss. Visualizations of classification results of all methods are shown in Figs. 5–7. It can be seen that the proposed method

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 16:03:31 UTC from IEEE Xplore. Restrictions apply.

TABLE IV

PERFORMANCE ON PAVIAU DATASET. THE RESULTS FROM THE BEST QUANTIZED METHOD IS COLORED IN RED AND THE SUBOPTIMAL IS COLORED IN BLUE. THE *n*w*m*a DENOTES QUANTIZATION WITH n-BIT WEIGHTS AND m-BIT ACTIVATIONS. THE MULTIPLIER OF (1w1a) DENOTES THE NUMBER OF PARALLEL BRANCHES

| Class | 1w1a | | | | | | | | | (1w1a)×5 | 1w2a | 2w2a | 32w32a |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| | XNOR | XNOR++ | Bi-Real Net | ReactNet | IRNet | ReCU | Real2Bin | EWGS | Ours | GroupNet | Mei | LSQ | Full Precision |
| 1 | 34.37 | 11.45 | 77.37 | 74.78 | 80.31 | 82.46 | 82.91 | 64.71 | 84.68 | 81.96 | 76.68 | 84.27 | 83.00 |
| 2 | 61.19 | 40.40 | 77.64 | 76.47 | 83.03 | 77.65 | 78.95 | 68.15 | 90.87 | 82.71 | 80.13 | 90.25 | 87.93 |
| 3 | 59.89 | 64.57 | 77.32 | 79.03 | 79.89 | 79.61 | 77.61 | 75.53 | 83.36 | 81.41 | 78.77 | 82.82 | 81.43 |
| 4 | 91.18 | 92.29 | 92.41 | 92.71 | 91.45 | 93.01 | 92.52 | 93.45 | 94.26 | 93.51 | 90.91 | 93.16 | 91.90 |
| 5 | 95.24 | 97.95 | 99.45 | 99.63 | 99.71 | 99.35 | 98.96 | 98.60 | 99.60 | 99.68 | 99.20 | 99.58 | 99.40 |
| 6 | 41.64 | 51.89 | 82.69 | 80.76 | 80.14 | 83.48 | 73.44 | 73.72 | 92.26 | 79.46 | 79.37 | 89.21 | 85.53 |
| 7 | 77.08 | 72.30 | 89.19 | 87.98 | 93.05 | 93.48 | 85.92 | 78.99 | 95.94 | 89.58 | 91.60 | 95.59 | 93.72 |
| 8 | 46.42 | 33.09 | 72.14 | 70.44 | 77.94 | 78.59 | 73.27 | 57.18 | 80.84 | 75.29 | 75.23 | 81.29 | 84.00 |
| 9 | 51.13 | 73.78 | 91.58 | 90.64 | 92.53 | 92.21 | 88.79 | 87.51 | 95.56 | 91.03 | 92.17 | 95.21 | 92.51 |
| OA(%) | 56.84 | 44.90 | 80.07 | 78.83 | 83.29 | 81.80 | 80.36 | 71.16 | 89.61 | 83.05 | 81.10 | 88.83 | 87.13 |
| AA(%) | 62.01 | 59.74 | 84.42 | 83.60 | 86.45 | 86.65 | 83.60 | 77.54 | 90.82 | 86.06 | 84.90 | 90.15 | 88.82 |
| KAPPA(%) | 46.14 | 34.65 | 74.53 | 73.04 | 78.37 | 76.71 | 74.71 | 63.89 | 86.41 | 78.05 | 75.70 | 85.37 | 83.20 |

TABLE V

PERFORMANCE ON SALINAS DATASET. THE RESULTS FROM THE BEST QUANTIZED METHOD IS COLORED IN RED AND THE SUBOPTIMAL IS COLORED IN BLUE. THE *n*w*m*a DENOTES QUANTIZATION WITH n-BIT WEIGHTS AND m-BIT ACTIVATIONS. THE MULTIPLIER OF (1w1a) DENOTES THE NUMBER OF PARALLEL BRANCHES

| Class | 1w1a | | | | | | | | | (1w1a)×5 | 1w2a | 2w2a | 32w32a |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| | XNOR | XNOR++ | Bi-Real Net | ReactNet | IRNet | ReCU | Real2Bin | EWGS | Ours | GroupNet | Mei | LSQ | Full Precision |
| 1 | 97.38 | 96.04 | 90.40 | 78.47 | 99.15 | 92.08 | 95.13 | 98.14 | 98.67 | 97.75 | 96.91 | 77.80 | 99.51 |
| 2 | 95.55 | 94.45 | 78.13 | 62.44 | 95.41 | 95.43 | 94.37 | 97.14 | 95.83 | 94.63 | 94.29 | 39.21 | 96.78 |
| 3 | 94.22 | 94.57 | 72.19 | 31.52 | 96.18 | 94.33 | 94.38 | 98.92 | 99.19 | 95.75 | 96.09 | 70.18 | 98.51 |
| 4 | 96.39 | 97.09 | 83.06 | 87.17 | 96.51 | 96.08 | 96.42 | 97.08 | 97.12 | 96.85 | 96.29 | 84.52 | 96.88 |
| 5 | 91.67 | 93.27 | 77.85 | 54.39 | 93.50 | 92.96 | 95.48 | 95.22 | 96.95 | 94.98 | 94.72 | 61.13 | 94.52 |
| 6 | 99.43 | 99.06 | 95.83 | 78.26 | 99.87 | 99.17 | 99.75 | 99.92 | 99.97 | 99.84 | 99.86 | 74.81 | 99.84 |
| 7 | 98.53 | 98.20 | 84.18 | 69.55 | 98.59 | 98.49 | 98.67 | 98.93 | 99.17 | 98.63 | 97.86 | 42.50 | 98.88 |
| 8 | 73.51 | 72.44 | 61.27 | 57.51 | 70.48 | 69.88 | 75.15 | 73.96 | 77.56 | 71.27 | 74.08 | 54.12 | 74.66 |
| 9 | 97.73 | 96.90 | 80.85 | 60.09 | 97.34 | 94.65 | 97.76 | 98.02 | 97.99 | 97.02 | 96.05 | 77.53 | 97.75 |
| 10 | 91.45 | 90.99 | 72.95 | 44.52 | 93.39 | 90.94 | 92.29 | 94.01 | 93.63 | 93.35 | 92.42 | 67.23 | 92.47 |
| 11 | 97.04 | 95.38 | 76.59 | 53.77 | 96.95 | 91.55 | 96.49 | 99.03 | 99.09 | 96.91 | 94.61 | 67.55 | 99.24 |
| 12 | 98.04 | 98.42 | 55.58 | 23.81 | 99.51 | 97.70 | 99.15 | 99.69 | 99.93 | 98.71 | 98.27 | 57.28 | 99.25 |
| 13 | 99.08 | 99.03 | 93.41 | 79.26 | 99.03 | 98.41 | 99.39 | 99.75 | 99.80 | 99.35 | 98.88 | 90.72 | 98.87 |
| 14 | 97.09 | 95.83 | 92.93 | 76.48 | 97.50 | 97.06 | 97.49 | 97.86 | 98.97 | 97.12 | 97.53 | 79.66 | 98.41 |
| 15 | 83.81 | 82.44 | 70.95 | 54.12 | 87.99 | 82.48 | 85.10 | 87.95 | 90.04 | 87.46 | 86.25 | 74.18 | 84.96 |
| 16 | 95.70 | 96.34 | 80.52 | 60.08 | 97.53 | 94.80 | 94.51 | 97.04 | 98.14 | 97.27 | 96.43 | 62.93 | 98.10 |
| OA(%) | 75.44 | 59.19 | 89.87 | 89.24 | 90.24 | 88.21 | 90.27 | 64.35 | 91.04 | 90.22 | 90.50 | 91.44 | 92.56 |
| AA(%) | 79.17 | 60.72 | 94.16 | 93.78 | 94.93 | 92.88 | 94.41 | 67.58 | 95.54 | 94.81 | 94.47 | 95.79 | 96.38 |
| KAPPA(%) | 72.95 | 55.10 | 88.74 | 88.05 | 89.17 | 86.92 | 89.20 | 60.66 | 90.04 | 89.14 | 89.45 | 90.49 | 91.74 |

can produce a classification visualization map that is more closer to what is generated from its counterpart full-precision model or ground truth. In summary, our proposed BNN for HSI classification maintains the accuracy of the corresponding full-precision model as much as possible and is more robust.

*2) Memory and Computation Cost Comparison:* The comparison of memory usage and floating point operations per second (FLOPs) of all methods is shown as Table VII. Following the calculation method of [41], the memory usage metric is the multiplication result of the parameter amounts and bitwidth, and the FLOPs of quantized convolution operator is calculated as $((c_{\text{in}} \times c_{\text{out}} \times k \times l_{\text{out}} \times b_a \times b_w)/64)$, where $c_{\text{out}}$ is the channel number of output feature maps, $c_{\text{in}}$ is the channel number of input feature maps, $k$ is the kernel size of 1-D-convolution, $l_{\text{out}}$ is the length of output feature maps, $b_a$ is the bit-width of activations, and $b_w$ is the bit-width of weights. All results shown in Table VII are obtained on the PaviaU dataset. Among all comparison methods, Real2Bin adds extra full-precision computation and memory usage due to the data-driven CR module, which is similar to the attention mechanism. Some computation and memory cost are also added due to the parallel binarization convolution in GroupNet or the 2-bit activations in Mei and LSQ. Though a full-precision adaptive gradient scale module is also plugged into the network during training stage in our method, it can be removed for inference without affecting the accuracy. Therefore, the computation and memory cost of our method during inference stage is completely equivalent to that of baseline Bi-Real Net.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 16:03:31 UTC from IEEE Xplore. Restrictions apply.

TABLE VI

PERFORMANCE ON HOUSTON DATASET. THE RESULTS FROM THE BEST QUANTIZED METHOD IS COLORED IN RED AND THE SUBOPTIMAL IS COLORED IN BLUE. THE *nwm*a DENOTES QUANTIZATION WITH n-BIT WEIGHTS AND m-BIT ACTIVATIONS. THE MULTIPLIER OF (1w1a) DENOTES THE NUMBER OF PARALLEL BRANCHES

| Class | 1w1a | | | | | | | | | (1w1a)×5 | 1w2a | 2w2a | 32w32a |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| | XNOR | XNOR++ | Bi-Real Net | ReactNet | IRNet | ReCU | Real2Bin | EWGS | Ours | GroupNet | Mei | LSQ | Full Precision |
| 1 | 73.02 | 48.61 | 92.16 | 92.79 | 93.58 | 93.26 | 93.30 | 82.28 | 91.68 | 92.17 | 94.56 | 95.03 | 94.94 |
| 2 | 90.07 | 58.27 | 96.82 | 96.94 | 97.41 | 98.52 | 97.26 | 91.77 | 97.21 | 96.52 | 98.05 | 97.29 | 97.41 |
| 3 | 93.45 | 43.35 | 98.52 | 98.61 | 98.52 | 98.98 | 98.76 | 95.60 | 98.33 | 98.16 | 98.64 | 98.21 | 98.59 |
| 4 | 75.47 | 29.67 | 89.77 | 85.44 | 92.19 | 89.67 | 91.68 | 74.07 | 94.60 | 90.35 | 93.24 | 93.22 | 97.30 |
| 5 | 94.57 | 68.00 | 99.24 | 99.14 | 99.46 | 99.44 | 99.41 | 98.68 | 99.26 | 99.05 | 99.43 | 99.25 | 99.43 |
| 6 | 85.20 | 52.26 | 94.91 | 94.29 | 96.40 | 96.22 | 95.64 | 92.04 | 96.58 | 95.16 | 96.73 | 96.69 | 97.75 |
| 7 | 64.49 | 39.61 | 75.68 | 72.10 | 74.67 | 73.36 | 74.03 | 67.27 | 78.33 | 71.41 | 75.20 | 75.63 | 76.82 |
| 8 | 51.19 | 32.39 | 74.43 | 73.62 | 77.07 | 75.50 | 77.36 | 62.58 | 80.07 | 74.56 | 75.94 | 80.97 | 82.97 |
| 9 | 53.13 | 29.67 | 71.98 | 71.08 | 74.44 | 72.96 | 73.59 | 59.44 | 76.52 | 73.09 | 73.36 | 79.04 | 78.64 |
| 10 | 63.01 | 29.22 | 81.30 | 79.34 | 84.48 | 81.42 | 84.26 | 71.22 | 87.12 | 82.67 | 83.27 | 85.85 | 87.39 |
| 11 | 51.40 | 26.90 | 77.60 | 75.98 | 79.20 | 77.08 | 83.06 | 66.51 | 84.88 | 75.59 | 78.96 | 84.83 | 92.17 |
| 12 | 43.43 | 19.08 | 75.06 | 73.44 | 79.73 | 77.20 | 80.07 | 58.55 | 81.34 | 75.88 | 80.41 | 84.68 | 86.08 |
| 13 | 72.72 | 52.53 | 88.78 | 87.40 | 89.52 | 88.00 | 90.69 | 85.66 | 92.51 | 87.35 | 90.76 | 94.15 | 94.39 |
| 14 | 95.34 | 50.64 | 98.60 | 98.47 | 98.25 | 98.12 | 98.07 | 96.14 | 98.31 | 98.33 | 98.81 | 98.41 | 99.02 |
| 15 | 98.93 | 71.49 | 99.56 | 99.44 | 99.90 | 99.92 | 99.38 | 99.61 | 99.72 | 99.57 | 99.92 | 99.92 | 99.95 |
| OA(%) | 70.01 | 40.87 | 85.56 | 84.32 | 87.13 | 85.95 | 87.30 | 76.72 | 88.78 | 87.22 | 85.27 | 89.23 | 90.74 |
| AA(%) | 73.70 | 43.45 | 87.63 | 86.54 | 88.99 | 87.98 | 89.12 | 80.09 | 90.43 | 89.15 | 87.32 | 90.88 | 92.19 |
| KAPPA(%) | 67.67 | 36.40 | 84.39 | 83.05 | 86.08 | 84.81 | 86.27 | 74.87 | 87.86 | 86.17 | 84.07 | 88.35 | 89.98 |

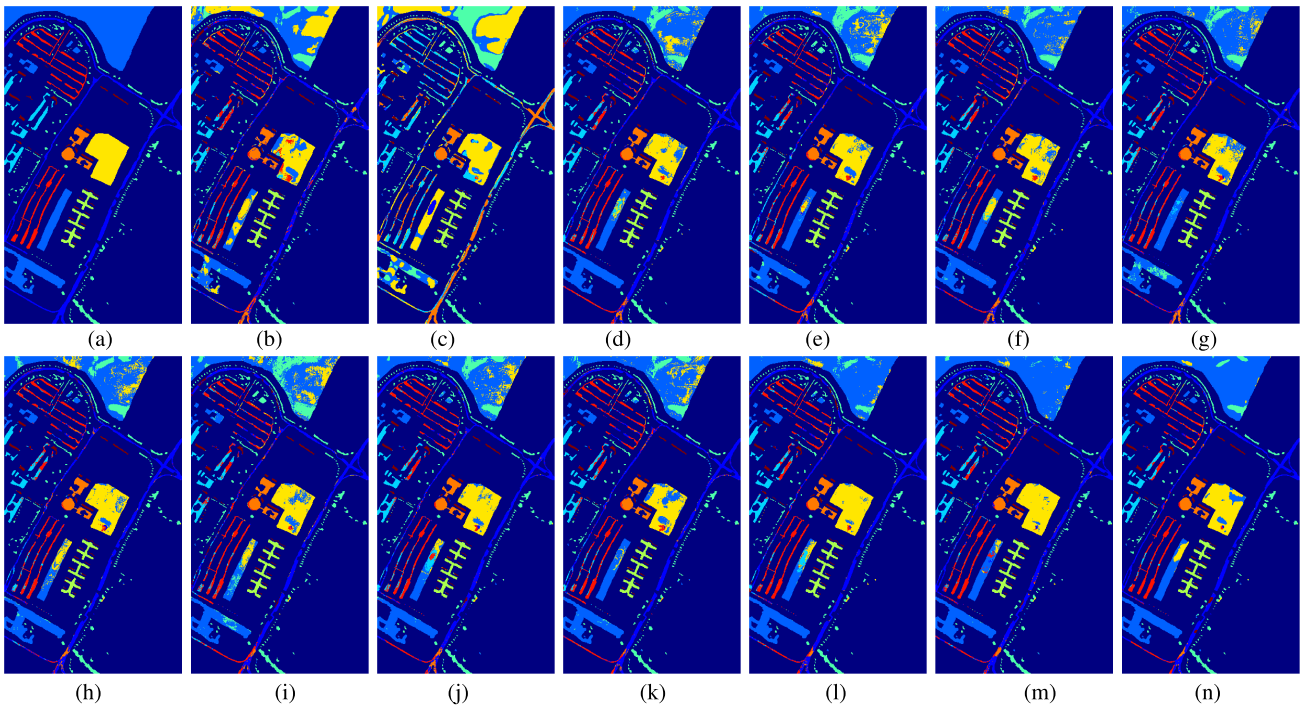



Fig. 5. Visual results of classification map on PaviaU Dataset. (a) GT. (b) XNOR. (c) XNOR++. (d) Bi-Real Net. (e) ReactNet. (f) IRNet. (g) ReCU. (h) Real2Bin. (i) EWGS. (j) GroupNet. (k) Mei. (l) LSQ. (m) Ours. (n) Full precision.

### C. Ablation Study

*1) Effectiveness of Individual Module:* In this section, we explore the effect of individual module on three datasets. As shown in Table VIII, the adaptive gradient scale module and the pixel sample-wise curriculum learning strategy both improve the accuracy of BNN. For example, on Salinas dataset, the curriculum learning strategy improves the OA by 0.71%, while the adaptive gradient scale improves the OA by 0.68%, and the joint action of these two modules improves OA by 1.17%. In Fig. 8, we visualize the classification effects brought by different modules on Salinas dataset. As can be seen, the classification map under the joint action of the modules is more similar to the ground truth, which demonstrates the effectiveness of each independent module.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 16:03:31 UTC from IEEE Xplore. Restrictions apply.

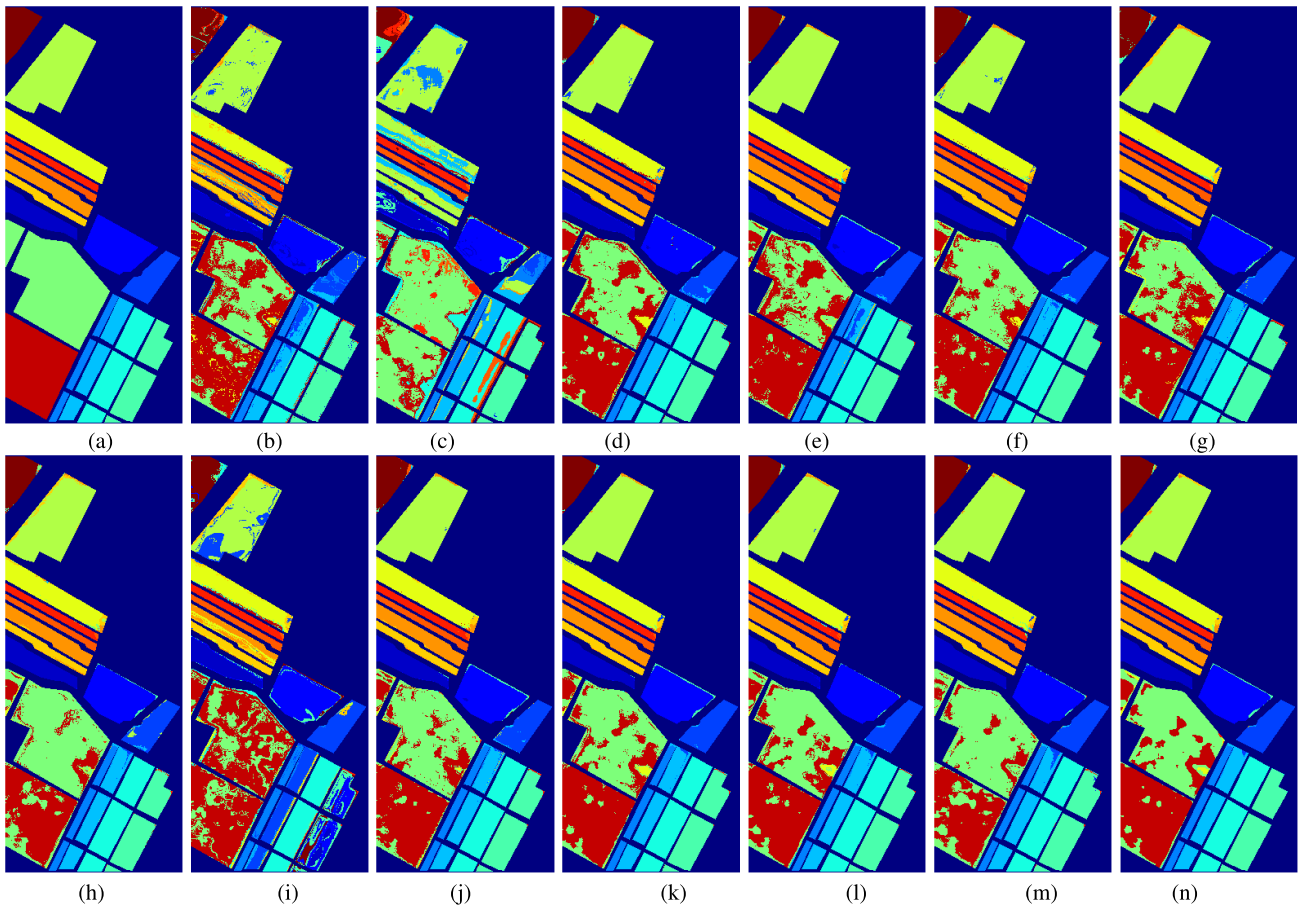



Fig. 6. Visual results of classification map on Salinas Dataset. (a) GT. (b) XNOR. (c) XNOR++. (d) Bi-Real Net. (e) ReactNet. (f) IRNet. (g) ReCU. (h) Real2Bin. (i) EWGS. (j) GroupNet. (k) Mei. (l) LSQ. (m) Ours. (n) Full precision.

TABLE VII

MEMORY USAGE AND COMPUTATION COMPARISON. W/A DENOTES THE BITWIDTH OF WEIGHTS AND ACTIVATIONS. THE MULTIPLIER OF (1/1) DENOTES THE NUMBER OF PARALLEL BRANCHES. SMALLER MEMORY USAGE AND FLOPS MEANS BETTER PERFORMANCE FOR RESOURCE-LIMITED OR LOW-LATENCY PLATFORMS. THE BEST RESULTS ARE BOLDED

| Methods | W/A | Memory usage(Mbit) | FLOPs |
|---|---|---|---|
| XNOR | 1/1 | 0.7759 | 238884 |
| XNOR++ | 1/1 | 0.8075 | 231906 |
| Bi-Real Net | 1/1 | 0.7759 | 236538 |
| ReactNet | 1/1 | 0.7846 | 264282 |
| IRNet | 1/1 | 0.7759 | 236538 |
| ReCU | 1/1 | 0.7846 | **221850** |
| Real2Bin | 1/1 | 1.0068 | 2253154 |
| EWGS | 1/1 | 0.7770 | 264282 |
| GroupNet | (1/1)×5 | 0.9468 | 714306 |
| Mei | 1/2 | 0.7759 | 331908 |
| LSQ | 2/2 | 0.8275 | 425442 |
| Ours | 1/1 | **0.7759** | 236538 |
| Full Precision | 32/32 | 2.0913 | 4514928 |

TABLE VIII

EFFECT OF EACH COMPONENT WITHIN THE PROPOSED METHOD ON THREE DATASETS (BL: BASELINE BNN; AGS: ADAPTIVE GRADIENT SCALE MODULE; SCL: SAMPLE-WISE CURRICULUM LEARNING STRATEGY). THE BEST RESULTS ARE BOLDED

| Dataset | BL | AGS | SCL | OA(%) | AA(%) | KAPPA(%) |
|---|---|---|---|---|---|---|
| PaviaU | √ | | | 80.07 | 84.42 | 74.53 |
| | √ | √ | | 80.64 | 85.19 | 75.21 |
| | √ | | √ | 88.86 | 90.80 | 85.47 |
| | √ | √ | √ | **89.61** | **90.82** | **86.41** |
| Salinas | √ | | | 89.87 | 94.16 | 88.74 |
| | √ | √ | | 90.55 | 94.57 | 89.50 |
| | √ | | √ | 90.58 | 95.52 | 89.54 |
| | √ | √ | √ | **91.04** | **95.54** | **90.04** |
| Houston | √ | | | 85.56 | 87.63 | 84.39 |
| | √ | √ | | 85.85 | 87.87 | 84.70 |
| | √ | | √ | 88.45 | 90.08 | 87.50 |
| | √ | √ | √ | **88.78** | **90.43** | **87.86** |

*2) Effect of the Way of Implementing the Adaptive Scale Gradient Module Structure:* There are many ways to implement the adaptive scale module structure. For example, it can be designed in the form of an attention module that performs on channel dim like the way of SENet [54] (denoted as SE), an attention module that performs on spectral band dim like the way of efficient channel attention (ECA)-Net [55] (denoted as Band), or an attention module that performs on both channel and spectral band dim like the way of convolutional block attention module (CBAM) [56] (denoted as

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 16:03:31 UTC from IEEE Xplore. Restrictions apply.

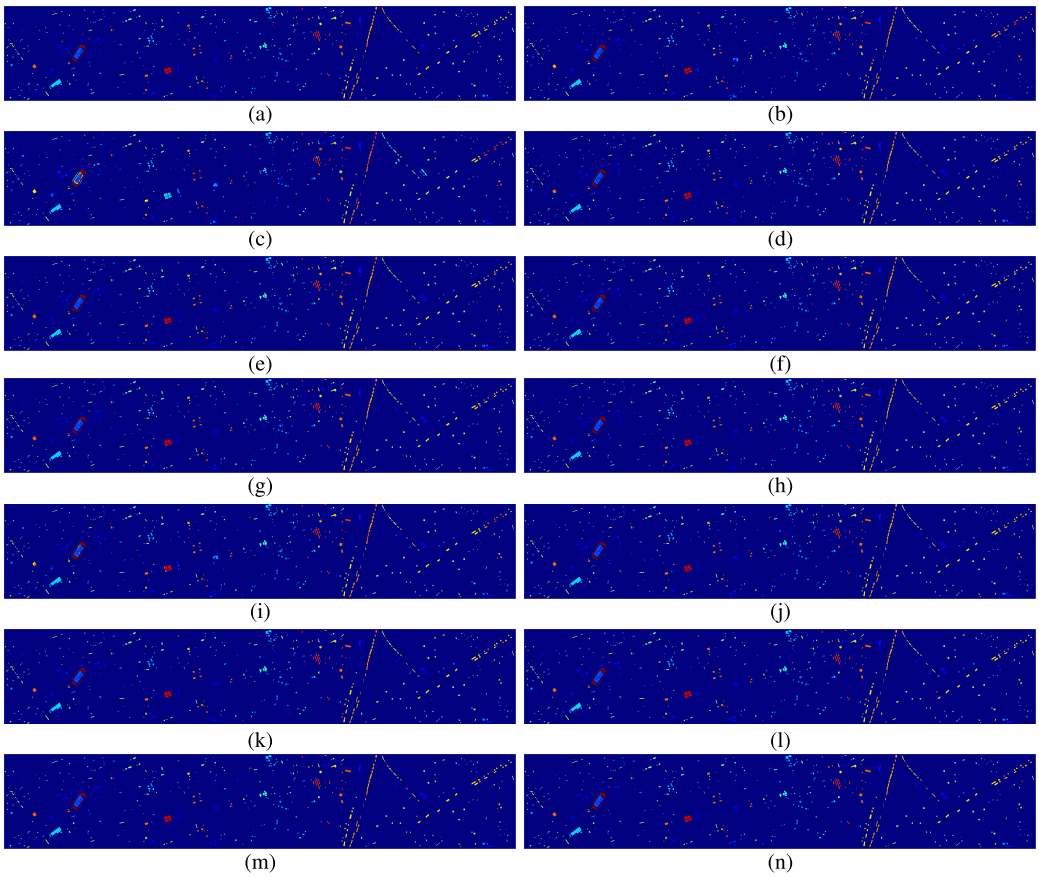



Fig. 7. Visual results of classification map on Houston Dataset. (a) GT. (b) XNOR. (c) XNOR++. (d) Bi-Real Net. (e) ReactNet. (f) IRNet. (g) ReCU. (h) Real2Bin. (i) EWGS. (j) GroupNet. (k) Mei. (l) LSQ. (m) Ours. (n) Full precision.

TABLE IX
EFFECT OF DIFFERENT IMPLEMENTATION OF AGS MODULE STRUCTURE. THE EXPERIMENT IS CONDUCTED ON PAVIAU DATASET. THE BEST RESULTS ARE BOLDED

| Methods | OA(%) | AA(%) | KAPPA(%) |
|---|---|---|---|
| Ours | **89.61** | 90.82 | **86.41** |
| SE | 89.28 | 90.71 | 85.99 |
| Mix | 89.05 | 90.47 | 85.68 |
| Band | 89.52 | **90.91** | 86.29 |

Mix). In our method, we construct the adaptive scale module structure utilizing an attention module performs on channel dim following the way of ECA-Net [55]. To clearly clarify the differences between the four implementation ways we tried, we provided their structures in Fig. 9. The experiment result on PaviaU dataset is shown as Table IX. It can be seen that our implementation achieves the highest accuracy. Attention performing on spectral band dim achieves suboptimal performance, while mixed attention CBAM has the worst effect. The reason behind may be that ECA-Net eliminates the dimension reduction process when generating the attention part compared with SE-Net and CBAM, thus avoiding the loss of accuracy.

*3) Effect of Convolution Kernel Size of Adaptive Scale Gradient Module:* The convolution kernel size of adaptive scale gradient module in Fig. 2 has an effect on accuracy. As the experiment result on PaviaU dataset shown in Table X,

TABLE X
EFFECT OF KERNEL SIZE OF CONVOLUTIONAL LAYER IN AGS MODULE. THE EXPERIMENT IS CONDUCTED ON PAVIAU DATASET. THE BEST RESULTS ARE BOLDED

| Kernel Size | OA(%) | AA(%) | KAPPA(%) |
|---|---|---|---|
| 3 | **89.61** | 90.82 | **86.41** |
| 5 | 89.48 | **90.92** | 86.28 |
| 7 | 88.82 | 90.75 | 85.44 |

the classification accuracy decreases with the increase of convolution kernel size and performs best when the kernel size equals to 3. Therefore, in all our experiments, we set the kernel size as 3.

*4) Effect of Sigmoid Function of Adaptive Scale Gradient Module:* The reason why the Sigmoid function is required is that it is necessary to maintain the same effect as when only using the sign function in the forward-propagation stage. To not increase any parameters and additional computation, the adaptive gradient scale module will be removed during inference stage. However, if Sigmoid function is not used for sign constraint, there would be differences for the sign of binarized activations between quantization-aware training and inferencing stage. After using the same weight parameters for convolution, different results will be obtained and thus will affect the final performance. To validate the conjecture, we conducted relevant comparison experiments with/without Sigmoid function within the adaptive gradient scale module.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 16:03:31 UTC from IEEE Xplore. Restrictions apply.

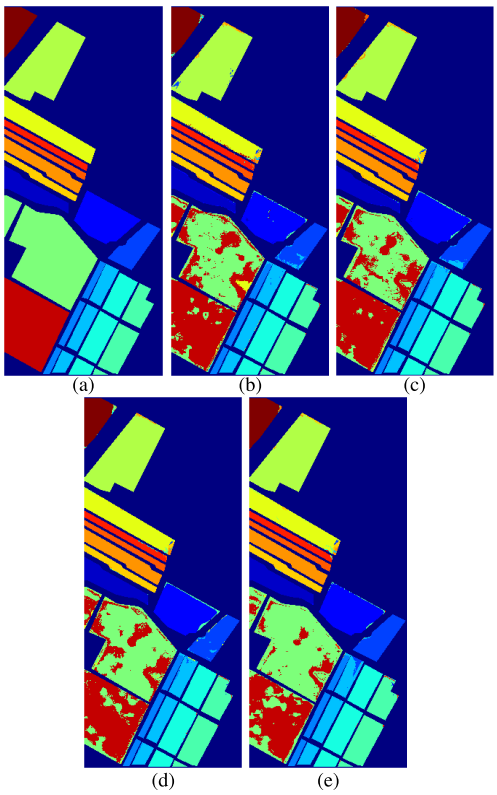



Fig. 8. Classification visual results of each component within the proposed method on Salinas dataset. (a) GT. (b) BL. (c) BL + AGS. (d) BL + SCL. (e) BL + AGS + SCL.

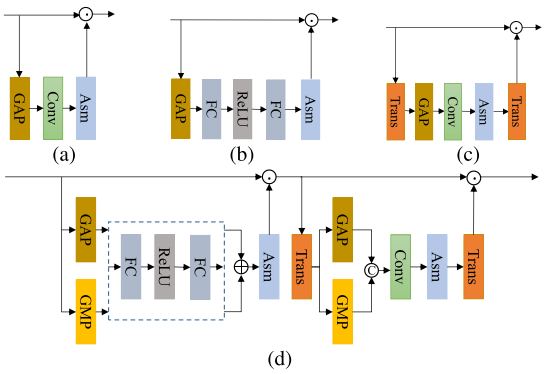



Fig. 9. Four ways to implement the adaptive gradient scale module. GAP and GMP denote the global average pooling and global maximum pooling layer, respectively, Asm denotes the Sigmoid function with two adjustable parameters, Trans denotes the matrix transpose operation, $\odot$ denotes element-wise multiplication, and © denotes the concatenation operation. (a) Ours. (b) SE [54]. (c) Band. (d) Mix [56].

The experimental results shown in Table XI demonstrate the necessity of Sigmoid function.

*5) Effect of Hyper-Parameter $\beta$ in Curriculum Learning:* Hyper-parameter $\beta$ controls the growth-rate of the importance value $z$. The larger $\beta$, the fewer iterations it takes for the sample to be given full attention. The experiment result on PaviaU dataset is shown as Fig. 10. It can be seen that too big or too small $\beta$ will bring an unfavorable impact on accuracy, which indicates that the importance value of $z$ should grow at an appropriate rate. In all our experiments, we set the $\beta$ as 0.15 following the default setting in [51].

TABLE XI
EFFECT OF SIGMOID FUNCTION ON PERFORMANCE. THE BEST RESULTS ARE BOLDED

| Dataset | Sigmoid | OA(%) | AA(%) | KAPPA(%) |
|---|---|---|---|---|
| PaviaU | √ | **89.61** | **90.82** | **86.41** |
| | × | 6.37 | 9.83 | -0.01 |
| Salinas | √ | **91.04** | **95.54** | **90.04** |
| | × | 5.99 | 6.74 | 0.01 |
| Houston | √ | **88.78** | **90.43** | **87.86** |
| | × | 6.60 | 7.48 | 0.01 |

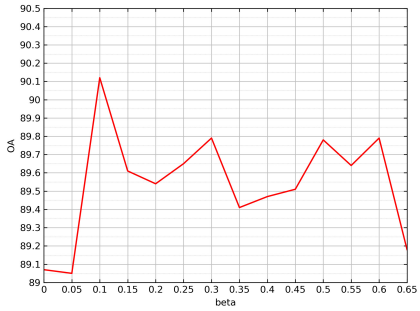



Fig. 10. Effect of $\beta$ in the adopted curriculum learning mechanism. The experiment is conducted on PaviaU dataset.

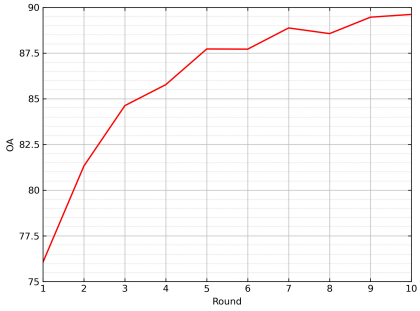



Fig. 11. Effect of the iteration rounds number in curriculum learning. The experiment is conducted on PaviaU dataset.

*6) Effect of the Iteration Rounds Number in Curriculum Learning:* As the number of rounds increases, more pixels challenging to be classified are fed into the network and the capacity and classification ability of the model improves gradually. The experiment result on PaviaU dataset shown as Fig. 11 indicates that as the iteration round increases, the accuracy generally increases. This is because the loss weights of pixel samples difficult to be classified are small at the beginning of the iterations, and the model only has the ability to correctly classify simple pixel samples. As the iteration round increases, the loss weights of pixel samples difficult to be classified become larger and larger, approximately equivalent to more and more difficult pixel samples are fed into the model to train, so the model gradually enhances its ability to deal with these difficult pixel samples.

## V. CONCLUSION

In this study, we propose a BNN with good performance for HSI classification task. To flexibly modify the back-propagated

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 16:03:31 UTC from IEEE Xplore. Restrictions apply.

gradient and better optimize the BNN, we first design an adaptive gradient scale module embedded in the network only during training stage, then a curriculum learning-based progressive binarization strategy is further adopted to make the training process get rid of suboptimal solution and further improve the performance. Experiment results on three datasets demonstrate the effectiveness of the proposed method, which significantly improves the accuracy of the BNN for HSI classification task without adding extra memory or computation cost for inference.

## REFERENCES


[1] S. Prasad and L. M. Bruce, "Limitations of principal components analysis for hyperspectral target recognition," *IEEE Geosci. Remote Sens. Lett.*, vol. 5, no. 4, pp. 625–629, Oct. 2008.

[2] D. Manolakis and G. S. Shaw, "Detection algorithms for hyperspectral imaging applications," *IEEE Signal Process. Mag.*, vol. 19, no. 1, pp. 29–43, Jan. 2002.

[3] H. Van Nguyen, A. Banerjee, and R. Chellappa, "Tracking via object reflectance using a hyperspectral video camera," in *Proc. IEEE Comput. Soc. Conf. Comput. Vis. Pattern Recognit. Workshops*, Jun. 2010, pp. 44–51.

[4] G. Licciardi, P. R. Marpu, J. Chanussot, and J. A. Benediktsson, "Linear versus nonlinear PCA for the classification of hyperspectral data based on the extended morphological profiles," *IEEE Geosci. Remote Sens. Lett.*, vol. 9, no. 3, pp. 447–451, May 2012.

[5] A. Villa, J. A. Benediktsson, J. Chanussot, and C. Jutten, "Hyperspectral image classification with independent component discriminant analysis," *IEEE Trans. Geosci. Remote Sens.*, vol. 49, no. 12, pp. 4865–4876, Dec. 2011.

[6] G. Camps-Valls, L. Gomez-Chova, J. Munoz-Mari, J. Vila-Frances, and J. Calpe-Maravilla, "Composite kernels for hyperspectral image classification," *IEEE Geosci. Remote Sens. Lett.*, vol. 3, no. 1, pp. 93–97, Jan. 2006.

[7] M. Fauvel, J. Chanussot, and J. A. Benediktsson, "A spatial–spectral kernel-based approach for the classification of remote-sensing images," *Pattern Recognit.*, vol. 45, no. 1, pp. 381–392, 2012.

[8] G. Rellier, X. Descombes, F. Falzon, and J. Zerubia, "Texture feature analysis using a Gauss-Markov model in hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 42, no. 7, pp. 1543–1551, Jul. 2004.

[9] F. Melgani and L. Bruzzone, "Classification of hyperspectral remote sensing images with support vector machines," *IEEE Trans. Geosci. Remote Sens.*, vol. 42, no. 8, pp. 1778–1790, Aug. 2004.

[10] P. K. Goel, S. O. Prasher, R. M. Patel, J.-A. Landry, R. Bonnell, and A. A. Viau, "Classification of hyperspectral data by decision trees and artificial neural networks to identify weed stress and nitrogen status of corn," *Comput. Electron. Agricult.*, vol. 39, no. 2, pp. 67–93, 2003.

[11] J. Ham, Y. Chen, M. M. Crawford, and J. Ghosh, "Investigation of the random forest framework for classification of hyperspectral data," *IEEE Trans. Geosci. Remote Sens.*, vol. 43, no. 3, pp. 492–501, Mar. 2005.

[12] E. Maggiori et al., "Convolutional neural networks for large-scale remote-sensing image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 55, no. 2, pp. 645–657, Feb. 2016.

[13] M. Längkvist, A. Kiselev, M. Alirezaie, and A. Loutfi, "Classification and segmentation of satellite orthoimagery using convolutional neural networks," *Remote Sens.*, vol. 8, no. 4, p. 329, 2016.

[14] Z. Deng, H. Sun, S. Zhou, J. Zhao, L. Lei, and H. Zou, "Multi-scale object detection in remote sensing imagery with convolutional neural networks," *ISPRS J. Photogramm. Remote Sens.*, vol. 145, pp. 3–22, Nov. 2018.

[15] X. Zheng, T. Gong, X. Li, and X. Lu, "Generalized scene classification from small-scale datasets with multitask learning," *IEEE Trans. Geosci. Remote Sens.*, vol. 60, 2022, Art. no. 5609311.

[16] W. Hu, Y. Huang, L. Wei, F. Zhang, and H. Li, "Deep convolutional neural networks for hyperspectral image classification," *J. Sensors*, vol. 2015, pp. 1–12, Jan. 2015.

[17] J. Yue, W. Zhao, S. Mao, and H. Liu, "Spectral–spatial classification of hyperspectral images using deep convolutional neural networks," *Remote Sens. Lett.*, vol. 6, no. 6, pp. 468–477, Jun. 2015.

[18] Y. Li, H. Zhang, and Q. Shen, "Spectral–spatial classification of hyperspectral imagery with 3D convolutional neural network," *Remote Sens.*, vol. 9, no. 1, p. 67, 2017.

[19] M. E. Paoletti, J. M. Haut, R. Fernandez-Beltran, J. Plaza, A. J. Plaza, and F. Pla, "Deep pyramidal residual networks for spectral–spatial hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 57, no. 2, pp. 740–754, Aug. 2018.

[20] G. Yang, U. B. Gewali, E. Ientilucci, M. Gartley, and S. T. Monteiro, "Dual-channel densenet for hyperspectral image classification," in *Proc. IEEE Int. Geosci. Remote Sens. Symp.*, Jul. 2018, pp. 2595–2598.

[21] Z. Gong, P. Zhong, Y. Yu, W. Hu, and S. Li, "A CNN with multiscale convolution and diversified metric for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 57, no. 6, pp. 3599–3618, Jun. 2019.

[22] J. Zhu, L. Fang, and P. Ghamisi, "Deformable convolutional neural networks for hyperspectral image classification," *IEEE Geosci. Remote Sens. Lett.*, vol. 15, no. 8, pp. 1254–1258, Aug. 2018.

[23] K. Pooja, R. R. Nidamanuri, and D. Mishra, "Multi-scale dilated residual convolutional neural network for hyperspectral image classification," in *Proc. 10th Workshop Hyperspectral Imag. Signal Process., Evol. Remote Sens. (WHISPERS)*, Sep. 2019, pp. 1–5.

[24] C. Wang, X. Bai, L. Zhou, and J. Zhou, "Hyperspectral image classification based on non-local neural networks," in *Proc. IEEE Int. Geosci. Remote Sens. Symp.*, Jul. 2019, pp. 584–587.

[25] L. Mou and X. X. Zhu, "Learning to pay attention on spectral domain: A spectral attention module-based convolutional network for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 58, no. 1, pp. 110–122, Sep. 2020.

[26] H. Sun, X. Zheng, X. Lu, and S. Wu, "Spectral–spatial attention network for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 58, no. 5, pp. 3232–3245, May 2020.

[27] R. Hang, Z. Li, Q. Liu, P. Ghamisi, and S. S. Bhattacharyya, "Hyperspectral image classification with attention-aided CNNs," *IEEE Trans. Geosci. Remote Sens.*, vol. 59, no. 3, pp. 2281–2293, Mar. 2021.

[28] X. Zheng, H. Sun, X. Lu, and W. Xie, "Rotation-invariant attention network for hyperspectral image classification," *IEEE Trans. Image Process.*, vol. 31, pp. 4251–4265, 2022.

[29] Y. Gong, L. Liu, M. Yang, and L. Bourdev, "Compressing deep convolutional networks using vector quantization," 2014, *arXiv:1412.6115*.

[30] K. Hwang and W. Sung, "Fixed-point feedforward deep neural network design using weights +1, 0, and −1," in *Proc. IEEE Workshop Signal Process. Syst. (SiPS)*, Oct. 2014, pp. 1–6.

[31] H. Li, A. Kadav, I. Durdanovic, H. Samet, and H. P. Graf, "Pruning filters for efficient ConvNets," 2016, *arXiv:1608.08710*.

[32] M. Lin et al., "HRank: Filter pruning using high-rank feature map," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2020, pp. 1529–1538.

[33] F. N. Iandola, S. Han, M. W. Moskewicz, K. Ashraf, W. J. Dally, and K. Keutzer, "SqueezeNet: AlexNet-level accuracy with 50x fewer parameters and <0.5MB model size," 2016, *arXiv:1602.07360*.

[34] A. G. Howard et al., "MobileNets: Efficient convolutional neural networks for mobile vision applications," 2017, *arXiv:1704.04861*.

[35] M. Jaderberg, A. Vedaldi, and A. Zisserman, "Speeding up convolutional neural networks with low rank expansions," 2014, *arXiv:1405.3866*.

[36] X. Cao, M. Ren, J. Zhao, H. Li, and L. Jiao, "Hyperspectral imagery classification based on compressed convolutional neural network," *IEEE Geosci. Remote Sens. Lett.*, vol. 17, no. 9, pp. 1583–1587, Sep. 2020.

[37] M. Courbariaux, I. Hubara, D. Soudry, R. El-Yaniv, and Y. Bengio, "Binarized neural networks: Training deep neural networks with weights and activations constrained to +1 or −1," 2016, *arXiv:1602.02830*.

[38] Y. Bengio, N. Léonard, and A. Courville, "Estimating or propagating gradients through stochastic neurons for conditional computation," 2013, *arXiv:1308.3432*.

[39] M. Rastegari, V. Ordonez, J. Redmon, and A. Farhadi, "XNOR-Net: ImageNet classification using binary convolutional neural networks," in *Proc. Eur. Conf. Comput. Vis.* Cham, Switzerland: Springer, 2016, pp. 525–542.

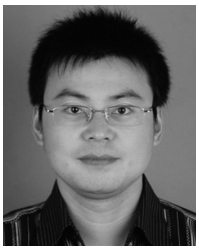
[40] A. Bulat and G. Tzimiropoulos, "XNOR-Net++: Improved binary neural networks," 2019, *arXiv:1909.13863*.

[41] B. Martinez, J. Yang, A. Bulat, and G. Tzimiropoulos, "Training binary neural networks with real-to-binary convolutions," 2020, *arXiv:2003.11535*.

[42] B. Zhuang, C. Shen, M. Tan, L. Liu, and I. Reid, "Structured binary neural networks for accurate image classification and semantic segmentation," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2019, pp. 413–422.


Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 16:03:31 UTC from IEEE Xplore. Restrictions apply.

[43] Z. Liu, B. Wu, W. Luo, X. Yang, W. Liu, and K.-T. Cheng, "Bi-Real Net: Enhancing the performance of 1-bit CNNs with improved representational capability and advanced training algorithm," in *Proc. Eur. Conf. Comput. Vis. (ECCV)*, Sep. 2018, pp. 722–737.
[44] Z. Liu, Z. Shen, M. Savvides, and K.-T. Cheng, "ReactNet: Towards precise binary neural network with generalized activation functions," in *Proc. Eur. Conf. Comput. Vis.* Cham, Switzerland: Springer, 2020, pp. 143–159.
[45] H. Qin et al., "Forward and backward information retention for accurate binary neural networks," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2020, pp. 2250–2259.
[46] R. Gong et al., "Differentiable soft quantization: Bridging full-precision and low-bit neural networks," in *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*, Oct. 2019, pp. 4852–4861.
[47] B. Zhuang, L. Liu, M. Tan, C. Shen, and I. Reid, "Training quantized neural networks with a full-precision auxiliary module," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2020, pp. 1488–1497.
[48] J. Lee, D. Kim, and B. Ham, "Network quantization with element-wise gradient scaling," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2021, pp. 6448–6457.
[49] S. Mei, X. Chen, Y. Zhang, J. Li, and A. Plaza, "Accelerating convolutional neural network-based hyperspectral image classification by step activation quantization," *IEEE Trans. Geosci. Remote Sens.*, vol. 60, 2021, Art. no. 5502012.
[50] B. Zhuang, M. Tan, J. Liu, L. Liu, I. Reid, and C. Shen, "Effective training of convolutional neural networks with low-bitwidth weights and activations," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 44, no. 10, pp. 6140–6152, Oct. 2022.
[51] L. Zhang et al., "Adaptive importance learning for improving lightweight image super-resolution network," *Int. J. Comput. Vis.*, vol. 128, no. 2, pp. 479–499, Feb. 2020.
[52] Z. Xu et al., "ReCU: Reviving the dead weights in binary neural networks," in *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*, Oct. 2021, pp. 5198–5208.
[53] S. K. Esser, J. L. McKinstry, D. Bablani, R. Appuswamy, and D. S. Modha, "Learned step size quantization," 2019, *arXiv:1902.08153*.
[54] J. Hu, L. Shen, and G. Sun, "Squeeze-and-excitation networks," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, Jun. 2018, pp. 7132–7141.
[55] Q. Wang, B. Wu, P. Zhu, P. Li, W. Zuo, and Q. Hu, "ECA-Net: Efficient channel attention for deep convolutional neural networks," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2020, pp. 13–19.
[56] S. Woo, J. Park, J.-Y. Lee, and I. S. Kweon, "CBAM: Convolutional block attention module," in *Proc. Eur. Conf. Comput. Vis. (ECCV)*, Sep. 2018, pp. 3–19.



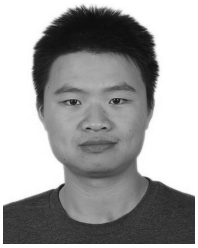
**Wei Wei** (Senior Member, IEEE) received the Ph.D. degree from Northwestern Polytechnical University, Xi'an, China, in 2012.

He is currently a Professor with the School of Computer Science and Engineering, Northwestern Polytechnical University. His research interests include image processing, machine learning, and pattern recognition. He has authored more than 40 papers including IEEE TRANSACTIONS ON GEOSCIENCE AND REMOTE SENSING, *International Journal of Computer Vision*, IEEE TRANSACTIONS ON IMAGE PROCESSING, Computer Vision and Pattern Recognition (CVPR), International Conference on Computer Vision (ICCV), Association for the Advancement of Artificial Intelligence (AAAI), and International Joint Conference on Artificial Intelligence (IJCAI).

Dr. Wei is a Reviewer for IEEE TRANSACTIONS ON GEOSCIENCE AND REMOTE SENSING, IEEE GEOSCIENCE AND REMOTE SENSING LETTERS, and IEEE TRANSACTIONS ON NEURAL NETWORKS AND LEARNING SYSTEMS. He is an Associate Editor of IEEE JOURNAL OF SELECTED TOPICS IN APPLIED EARTH OBSERVATIONS AND REMOTE SENSING.



**Chongxing Song** received the B.E. degree from Northwestern Polytechnical University, Xi'an, China, in 2021, where he is currently pursuing the M.E. degree.



**Lei Zhang** (Member, IEEE) received the Ph.D. degree in computer science and technology from Northwestern Polytechnical University, Xi'an, China, in 2018.

He was a Research Staff with the School of Computer Science, The University of Adelaide, Adelaide, SA, Australia, from 2017 to 2019. He was a Research Scientist with the Inception Institute of Artificial Intelligence (IIAI), Abu Dhabi, United Arab Emirates, from 2019 to 2020. He is currently a Professor with the School of Computer Science, Northwestern Polytechnical University. His research interests include image processing, machine learning, and video analysis.



**Yanning Zhang** (Senior Member, IEEE) received the B.S. degree from the Dalian University of Technology, Dalian, China, in 1988, the M.S. degree from the School of Electronic Engineering, Northwestern Polytechnical University, Xi'an, China, in 1993, and the Ph.D. degree from the School of Marine Engineering, Northwestern Polytechnical University, in 1996.

She is currently a Professor with the School of Computer Science, Northwestern Polytechnical University. She is also a Cheung Kong Professor of the Ministry of Education, China. She has authored more than 200 papers. Her research interests include remote sensing image analysis, computer vision, and pattern recognition.

Dr. Zhang is an Associate Editor of IEEE TRANSACTIONS ON GEOSCIENCE AND REMOTE SENSING, IEEE TRANSACTIONS ON MULTIMEDIA, and *Pattern Recognition*.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 06,2026 at 16:03:31 UTC from IEEE Xplore. Restrictions apply.